

SecureSync - Secure Data Exchange Platform

A military-grade secure data synchronization system for distributed manufacturing plants with end-to-end encryption, digital signatures, and real-time threat monitoring.

Overview

SecureSync solves critical security challenges for manufacturing companies with distributed plants:

- Data breaches during transmission
- No proof of data origin
- DDoS attacks on API endpoints
- Unauthorized access attempts
- Data tampering without detection
- Replay attacks

Solution: Comprehensive security combining AES-256-GCM encryption, RSA-2048 signatures, IP whitelisting, rate limiting, and forensic audit trails.

Core Security Strengths

1. End-to-End Encryption (25%)

Files: `crypto.py`, `plant_send.py`, `main.py`

AES-256-GCM Encryption

- 256-bit keys (`os.urandom(32)`)
- Authenticated encryption prevents tampering
- Function: `crypto.py` → `aes_encrypt()` / `aes_decrypt()`

RSA-2048 Key Exchange

- OAEP padding with SHA-256
- Prevents chosen-ciphertext attacks
- Function: `crypto.py` → `rsa_encrypt()` / `rsa_decrypt()`

Large Payload Support

- Up to 50MB (`config.py`) → (`MAX_PAYLOAD_SIZE`)
- Validated in (`main.py`) line 58

2. Digital Signatures & Non-Repudiation (25%)

Files: (`crypto.py`), (`plant_send.py`), (`main.py`)

RSA-PSS Signatures

- Probabilistic Signature Scheme with SHA-256
- Maximum salt length for security
- Function: (`crypto.py`) → (`sign()`) / (`verify_signature()`)

Verification Process

- Signature computed on plaintext (not ciphertext)
- Verified after decryption ((`main.py`) line 69-72)
- Any tampering invalidates signature

Audit Trail

- Database: (`audit_logs`) table
- Records: `plant_id`, `payload_hash` (SHA-256), `source_ip`, `timestamp`
- Proves: "Plant A sent this exact data at this time"

3. IP Whitelisting (20%)

Files: (`database.py`), (`security.py`), (`main.py`)

Multi-Layer Defense

- Layer 1: Middleware blocks before endpoints ((`security.py`))
- Layer 2: Endpoint validation ((`main.py`) line 39-41)
- Database: (`whitelist_ips`) table with PostgreSQL

Dynamic Management

- Functions: (`add_ip_to_whitelist()`), (`remove_ip_from_whitelist()`)
- Admin UI: React dashboard for runtime changes

- Attack logging: All unauthorized attempts recorded

4. Rate Limiting & DDoS Protection (20%)

Files: `utils.py`, `ratelimit.py`, `security.py`, `config.py`

Sliding Window Algorithm

- Implementation: `utils.py` → `rate_limited()`
- Default: 20 requests/minute (`config.py`)
- In-memory cache for speed

Protection Mechanism

- Middleware enforcement before database queries
- Rate check happens BEFORE crypto operations
- Protects database from connection exhaustion
- Rate-limited requests: <1ms response time

5. Replay Attack Prevention

Files: `database.py`, `main.py`

Nonce-Based Protection

- Each request has unique UUID nonce
- Database PRIMARY KEY constraint ensures atomicity
- Stored BEFORE decryption (`main.py` line 52)

Time Window

- 5-minute replay window (`config.py` → `REPLAY_WINDOW=300`)
- Old nonces auto-cleaned via `cleanup_old_nonces()`

Timestamp Validation

- Clock drift tolerance: ± 30 seconds
- Prevents old message replay (`main.py` line 45-47)

6. Admin Authentication

Files: `main.py`, `api.js`, `App.js`

Firebase Integration

- JWT token verification via Firebase Admin SDK
- All `/admin/*` endpoints require authentication
- Frontend: Automatic token attachment (`api.js` line 12-19)

7. Forensic Audit System

Files: `database.py`, `main.py`

Dual Logging

- `audit_logs`: Successful transmissions
- `attack_logs`: Security violations

Attack Types Logged

- `UNAUTHORIZED_IP`, `INVALID_API_KEY`, `STALE_REQUEST`
- `REPLAY_ATTACK`, `INVALID_SIGNATURE`, `RATE_LIMIT`
- `DECRYPTION_FAILED`, `PAYLOAD_TOO_LARGE`

Optimizations

- Time-based indexes for fast queries
- 30-day retention with auto-cleanup
- Real-time visualization in React dashboard

Technology Stack

Backend: FastAPI, PostgreSQL, Cryptography, Firebase Admin SDK

Frontend: React, Recharts, Axios, Firebase Auth, Lucide Icons

Security: AES-256-GCM, RSA-2048-OAEP, RSA-PSS, SHA-256

Installation & Setup

Backend Setup

```

bash
    
```

```
pip install -r requirements.txt
export DB_PASSWORD="your_password"
python setup_database.py
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Frontend Setup

```
bash

cd frontend
npm install
echo "VITE_API_URL=http://localhost:8000" > .env
npm run dev
```

Firebase Configuration

1. Create Firebase project
 2. Enable Email/Password authentication
 3. Download `firebase-adminsdk.json` to backend root
 4. Add Firebase config to `frontend/src/firebase.js`
-

API Documentation

POST /sync

Receive encrypted data from plants

Authentication: IP whitelist + API key + signature

Rate Limit: 20 req/min per IP

Request:

```
json
```

```
{
  "plant_id": "PLANT_A",
  "api_key": "abc123",
  "nonce_id": "uuid",
  "timestamp": 1706140800,
  "encrypted_key": "base64_rsa_encrypted_key",
  "cipher": "base64_aes_encrypted_payload",
  "nonce": "base64_aes_nonce",
  "signature": "hex_rsa_signature"
}
```

Response:

```
json

{
  "status": "accepted",
  "payload_hash": "sha256_hash",
  "received_at": 1706140800
}
```

GET /admin/audit-logs

View successful transmissions (requires auth)

GET /admin/attack-logs

View security events (requires auth)

GET /admin/attack-stats?interval=minute

Time-series attack frequency (requires auth)

Performance Metrics

Metric	Value
Normal Request	80-100ms
Rate Limited Request	<1ms
Database Query	5-15ms

Metric	Value
AES Decrypt (10KB)	~0.5ms
AES Decrypt (50MB)	~150ms

Capacity: 500 req/s (legitimate), 50,000 req/s (rate-limited)

Future Prospects

Short-Term (1-3 Months)

- **Mutual TLS (mTLS):** Plants verify server identity
- **HMAC Integrity:** Additional layer over AES-GCM
- **Geolocation Validation:** Verify IP country matches plant location
- **Automated Alerts:** Email/Slack notifications for attacks
- **Certificate-Based Auth:** X.509 certificates instead of API keys

Medium-Term (3-6 Months)

- **Redis Rate Limiting:** Distributed rate limiting for horizontal scaling
- **Data Anonymization:** Format-preserving encryption for PII fields
- **WebSocket Dashboard:** Real-time streaming updates
- **ML Anomaly Detection:** Detect unusual patterns in audit logs
- **Blockchain Audit Trail:** Immutable proof via Ethereum/Hyperledger

Long-Term (6-12 Months)

- **Multi-Region Deployment:** Geographic distribution with Kubernetes
- **Zero-Trust Architecture:** Never trust, always verify
- **Quantum-Resistant Crypto:** CRYSTALS-Kyber/Dilithium migration
- **Automated Pen Testing:** CI/CD security scanning
- **Compliance Certification:** ISO 27001, SOC 2, GDPR

Research & Innovation

- **Homomorphic Encryption:** Analytics on encrypted data

- **Confidential Computing:** Secure enclaves (Intel SGX, AMD SEV)
 - **Differential Privacy:** Anonymized aggregate analytics
 - **AI-Powered Threat Hunting:** Deep learning for attack prediction
-

Security Testing

Test Rate Limiting:

```
bash

for i in {1..100}; do curl -X POST http://localhost:8000/sync; done
# Expected: First 20 pass, remaining 80 get 429
```

Test Replay Protection:

```
python

# Send same nonce twice - second request gets 409
```

Penetration Testing Checklist:

- IP blocking, rate limiting, replay attacks, timestamp validation
 - Signature tampering, payload size limits, SQL injection, admin auth
-

Project Structure

```
securesync/
├── backend/
│   ├── main.py      # FastAPI app & endpoints
│   ├── crypto.py    # Encryption/signature functions
│   ├── database.py  # PostgreSQL operations
│   ├── security.py  # Middleware (IP/rate limit)
│   ├── utils.py     # Rate limiting logic
│   ├── config.py    # Configuration constants
│   ├── setup_database.py # Database initialization
│   └── plant_send.py # Example plant client
├── frontend/
│   └── src/
│       └── App.js    # React dashboard
```

```
| | | — api.js    # API client
| | | — firebase.js # Firebase config
| | | — App.css    # Styles
| | — package.json
| — requirements.txt
```

Success Criteria Met

- ✓ **AES-256 encryption/decryption:** `crypto.py` with GCM mode
 - ✓ **Digital signatures verified:** RSA-PSS in `main.py` line 69-72
 - ✓ **IP whitelisting blocks unauthorized:** Middleware + endpoint checks
 - ✓ **Rate limiting prevents DDoS:** Sliding window, <1ms for blocked requests
 - ✓ **Complete audit trail:** `audit_logs` + `attack_logs` tables
 - ✓ **<100ms latency:** Optimized architecture with indexes
-
-

Built with security-first principles for enterprise manufacturing environments.